Tim Farnung, Dieter Grünke, Erik Sauer, Leonhard Schneider

Projekt: StudyConnect

Lab 8

Part A: Manual CVE Hunt

List of direct dependencies

The list of direct dependencies was created using the tool `pipreqs`. Since it is present in the requirements.txt, it is usable out of the box.

To explicitly avoid overwriting the existing requirements list, the command used was: `pipreqs --savepath direct_reqs.txt ./` (from directory `backend/`). Results see direct_reqs.txt

NOTEs:

- The version numbers do not match the one's used in the project's requirements.txt.
- The wrong keycloak package was detected and has to be manually corrected.

The corrected output with the respective package version is listed below:

```
behave==1.3.3
Flask==3.1.2
flask-cors==6.0.1
Flask-SQLAlchemy==3.1.1
python-keycloak==5.8.1
python-dotenv==1.1.1
SQLAlchemy==2.0.43
```

CVE Findings Table

The following table lists some of the critical vulnerabilities found in the project's dependencies. Each entry includes the CVE ID, CVSS score, affected version range, the version that fixes the issue, and the dependency type. All findings were initially looked up on NVD (nvd.nist.gov) and successfully cross-checked on OSV (osv.dev).

CVE ID	CVSS	Affected Version Range	Fix Version	Dep. Type
CVE-2026-28684	6.6	< 1.2.2	1.2.2	Direct
CVE-2026-44432	7.5	2.6.0 - 2.6.x	2.7.0	Transitive
CVE-2026-31958	7.5	< 6.5.5	6.5.5	Transitive
CVE-2026-44896	6.1	≤ 3.2.0	3.2.1	Transitive

Package List Excerpts

Below is an excerpt of the project's dependencies, annotated to show which packages were analyzed and whether vulnerabilities were found. Packages marked with ⚠️ have known CVEs, while ✅ indicates no issues were detected.

Package	Version	Status
flask	3.1.2	✅
python-dotenv	1.1.1	⚠️ CVE-2026-28684
flask-cors	6.0.1	✅
Flask-SQLAlchemy	3.1.1	✅
python-keycloak	5.8.1	✅
SQLAlchemy	2.0.43	✅
behave	1.3.3	✅
urllib3	2.6.0	⚠️ CVE-2026-44432, CVE-2026-44431
tornado	6.5.2	⚠️ CVE-2026-31958, CVE-2026-35536
mistune	3.1.4	⚠️ CVE-2026-44896, CVE-2026-44899

Part B: SBOM and Automated Scan

Generate SBOM

Install cdxgen with npm `npm install -g @cyclonedx/cdxgen' and run 'cdxgen --output sbom.json --json-pretty`, this generates the sbom file sbom.json.

Count components

Using `jq '.components | length' sbom.json` or `python3 -c "import json; data=json.load(open('backend/sbom.json')); print(len(data['components']))"` counts the amount of components in the sbom.json when in the backend folder.

The amount in this projects sbom is: **146 components**

Fields verification

The SBOM has the following structure:


```
{
"bomFormat": "CycloneDX",
"specVersion": "1.7",
"serialNumber": "urn:uuid:e351d086-896a-4a7a-8920-242921edda23",
"version": 1,
"metadata": {
    "timestamp": "2026-06-14T18:51:44Z",
    "tools": {},
```

```
      "component": {
          "group": "",
          "name": "backend",
          "version": "latest",
          "type": "application",
          "bom-ref": "pkg:pypi/backend@latest",
          "purl": "pkg:pypi/backend@latest"
      },
  },
  "components": [
      {
        "group": "",
        "name": "asttokens",
        "version": "3.0.0",
        "purl": "pkg:pypi/asttokens@3.0.0",
        "type": "library",
        "bom-ref": "pkg:pypi/asttokens@3.0.0",
        "properties": [],
        "evidence": {}
      }
  ],
```


This structure includes all important fields.

Scan results

install trivy and execute

```
trivy sbom sbom.cdx.json --output trivyScan.txt
```

The results are found in trivyScan.txt.

Manual VS Trivy Findings

The table below lists all CVEs found across both methods. The 4 CVEs from the manual hunt were all confirmed by Trivy; Trivy found 19 additional vulnerabilities not caught manually.

CVE ID	Package	Severity	Manual	Trivy	Fix Version
CVE-2026-28684	python-dotenv	MEDIUM	✅	✅	1.2.2
CVE-2026-44432	urllib3	HIGH	✅	✅	2.7.0
CVE-2026-31958	tornado	HIGH	✅	✅	6.5.5
CVE-2026-44896	mistune	MEDIUM	✅	✅	3.2.1
CVE-2026-27205	Flask	LOW	×	✅	3.1.3
CVE-2026-4539	Pygments	LOW	×	✅	2.20.0
CVE-2026-21860	Werkzeug	MEDIUM	×	✅	3.1.5

CVE ID	Package	Severity	Manual	Trivy	Fix Version
CVE-2026-27199	Werkzeug	MEDIUM	×	✅	3.1.6
CVE-2026-32274	black	HIGH	×	✅	26.3.1
CVE-2026-45409	idna	MEDIUM	×	✅	3.15
CVE-2026-33079	mistune	HIGH	×	✅	3.2.1
CVE-2026-44708	mistune	MEDIUM	×	✅	— (no fix)
CVE-2026-44897	mistune	MEDIUM	×	✅	3.2.1
CVE-2025-53000	nbconvert	HIGH	×	✅	7.17.0
CVE-2026-39377	nbconvert	MEDIUM	×	✅	7.17.1
CVE-2026-39378	nbconvert	MEDIUM	×	✅	7.17.1
CVE-2025-71176	pytest	MEDIUM	×	✅	9.0.3
CVE-2026-25645	requests	MEDIUM	×	✅	2.33.0
CVE-2026-35536	tornado	HIGH	×	✅	— (no fix)
GHSA-78cv-mqj4-43f7	tornado	MEDIUM	×	✅	— (no fix)
CVE-2026-49854	tornado	LOW	×	✅	6.5.6
CVE-2026-21441	urllib3	HIGH	×	✅	2.6.3
CVE-2026-44431	urllib3	HIGH	×	✅	2.7.0

Summary: The manual hunt identified 4 CVEs, all of which were confirmed by Trivy. Trivy found 23 vulnerabilities in total — 19 additional ones across packages such as `black`, `Werkzeug`, `nbconvert`, `pytest`, and `requests` that were not checked manually. This demonstrates the advantage of automated SBOM scanning: it covers the full transitive dependency tree systematically, whereas the manual approach was limited to a selected subset of packages.

VEX Statement

```
{
  "@context": "https://openvex.dev/ns/v0.2.0",
  "@id": "https://studyconnect.example.com/vex/CVE-2026-32274/1",
  "author": "Leonhard Schneider <studyconnect-sec@example.com>",
  "timestamp": "2026-06-14T18:00:00Z",
  "version": 1,
  "statements": [
    {
      "vulnerability": {
        "@id": "https://nvd.nist.gov/vuln/detail/CVE-2026-32274",
        "name": "CVE-2026-32274",
        "description": "HIGH-severity vulnerability in black (Python code
formatter) affecting versions prior to 26.3.1"
```

```
      },
      "products": [
        {
          "@id": "pkg:pypi/backend@latest",
          "identifiers": {
            "purl": "pkg:pypi/backend@latest"
          },
          "subcomponents": [
            {
              "@id": "pkg:pypi/black@25.1.0"
            }
          ]
        }
      ],
      "status": "not_affected",
      "justification": "vulnerable_code_not_in_execute_path",
      "impact_statement": "The StudyConnect backend includes black as a
transitive dependency pulled in solely by development and testing tooling
(e.g. pytest plugins). black is a Python source-code formatter and is never
invoked during application startup, request handling, or any production
code path. No mechanism exists by which an external attacker could trigger
the vulnerable formatting logic at runtime. The application does not expose
any interface that processes or formats Python source code. Therefore,
despite the presence of black in the SBOM, the vulnerable code path is
unreachable in the deployed product.",
      "evidence": [
        {
          "type": "code_review",
          "description": "Review of backend/ source code confirms that
black is not imported or called anywhere in the application code. It
appears only as a transitive dependency of dev/test tooling and is absent
from the application's import graph at runtime."
        },
        {
          "type": "automated_scan",
          "description": "Trivy SBOM scan (trivy sbom sbom.cdx.json)
against CycloneDX SBOM generated with cdxgen flagged CVE-2026-32274 as
HIGH. Scan output: backend/trivyScan.txt. The finding is present in the
SBOM because black is listed as a component, but runtime reachability
analysis rules out exploitability."
        },
        {
          "type": "sbom",
          "description": "CycloneDX SBOM (specVersion 1.7, serialNumber
urn:uuid:e351d086-896a-4a7a-8920-242921edda23) lists black among 146
components. SBOM generated 2026-06-14T18:51:44Z."
        }
      ]
    }
  ]
}
```